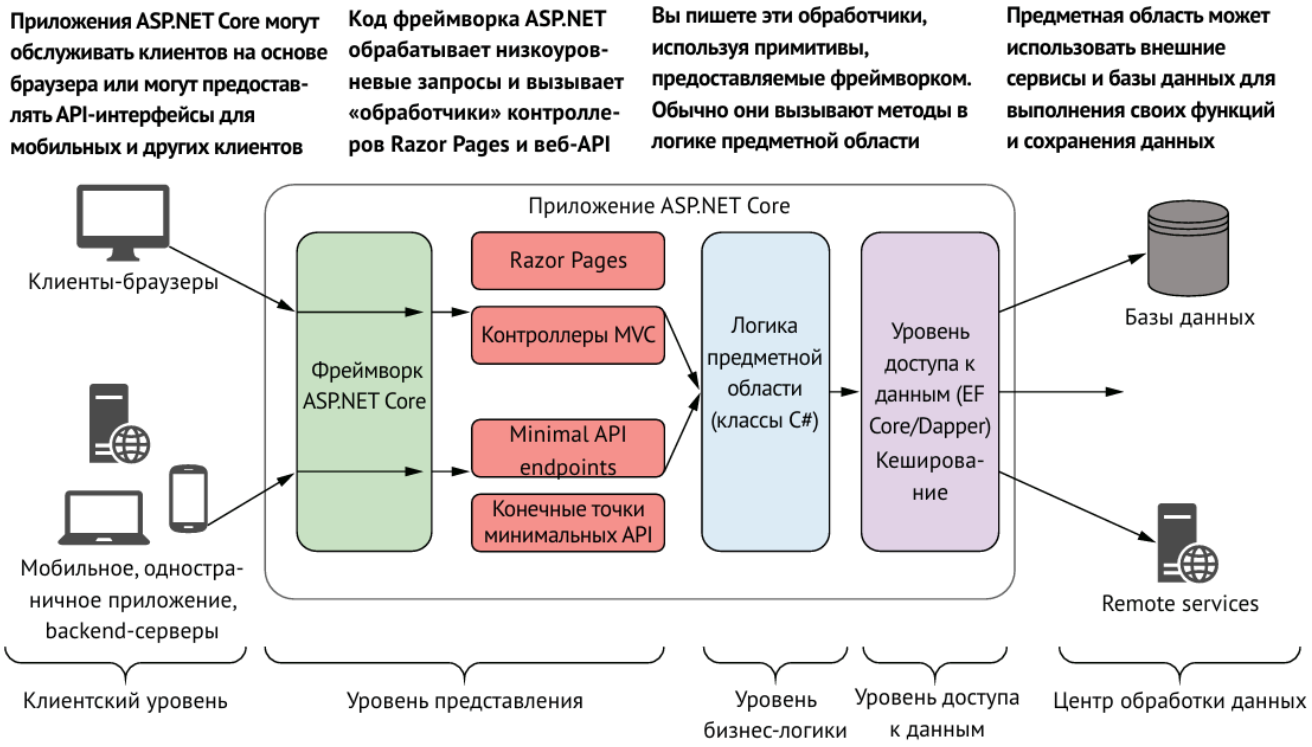
Введение в ASP.NET Core и Web API

Введение

ASP.NET Core — кроссплатформенный фреймворк с открытым исходным кодом, разработанный компанией Microsoft для создания высокопроизводительных веб-приложений и сервисов. Он работает на Windows, Linux и macOS и входит в экосистему .NET.

Фреймворк поддерживает создание различных типов приложений:

- **Веб-приложения** (MVC, Razor Pages) — серверный рендеринг HTML-страниц.
- **API** (RESTful, GraphQL) — предоставление данных для внешних клиентов.
- **Микросервисы** — небольшие независимые сервисы, взаимодействующие друг с другом.
- **Realtime-приложения** (SignalR) — чаты, уведомления, биржевые данные.



Как работает ASP.NET Core приложение

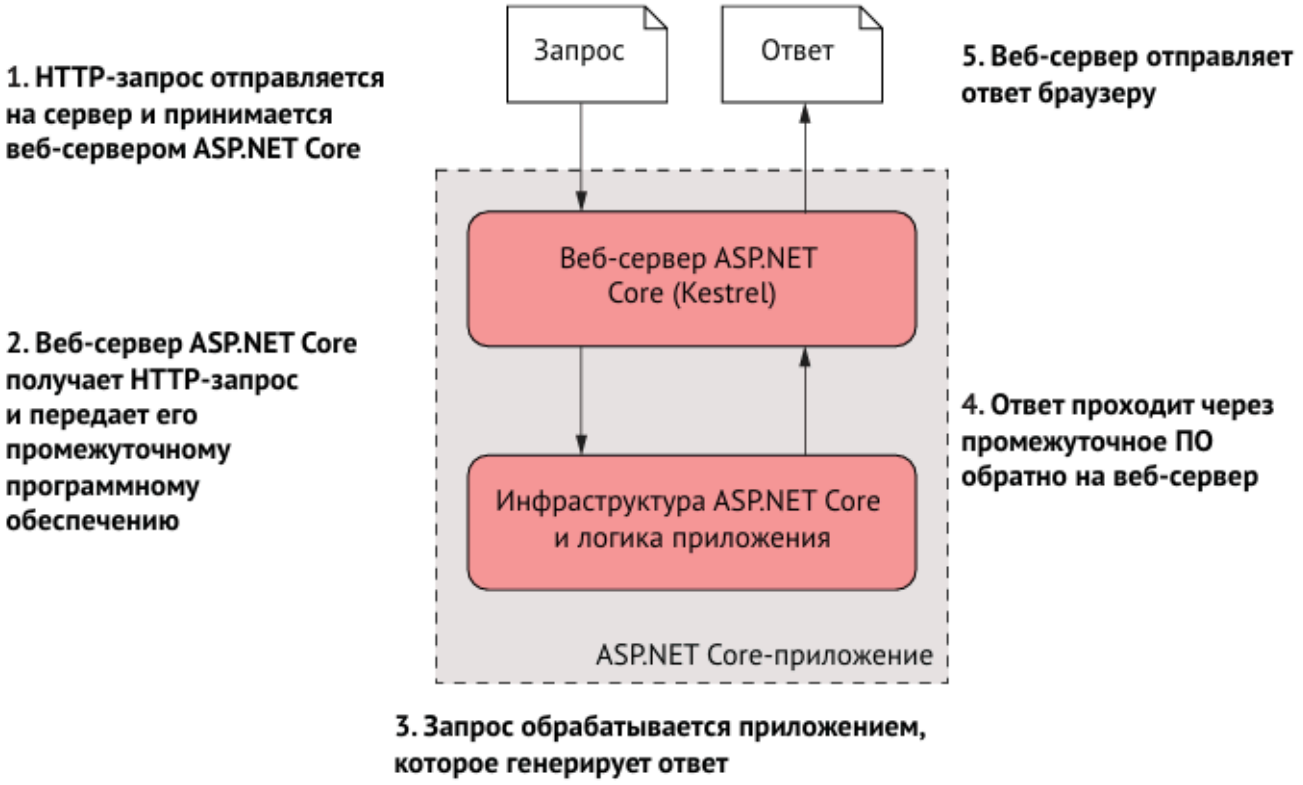
При запуске ASP.NET Core приложения происходит следующее: операционная система запускает процесс .NET, который поднимает встроенный веб-сервер **Kestrel** и начинает принимать входящие HTTP-запросы.

Kestrel — это кроссплатформенный веб-сервер, встроенный в ASP.NET Core. Он написан на C# и работает на всех платформах, поддерживаемых .NET. Именно Kestrel слушает сетевой порт, принимает TCP-соединения, разбирает HTTP-запросы и

передаёт их в приложение. По умолчанию он запускается на портах `5000` (HTTP) и `7000` (HTTPS).

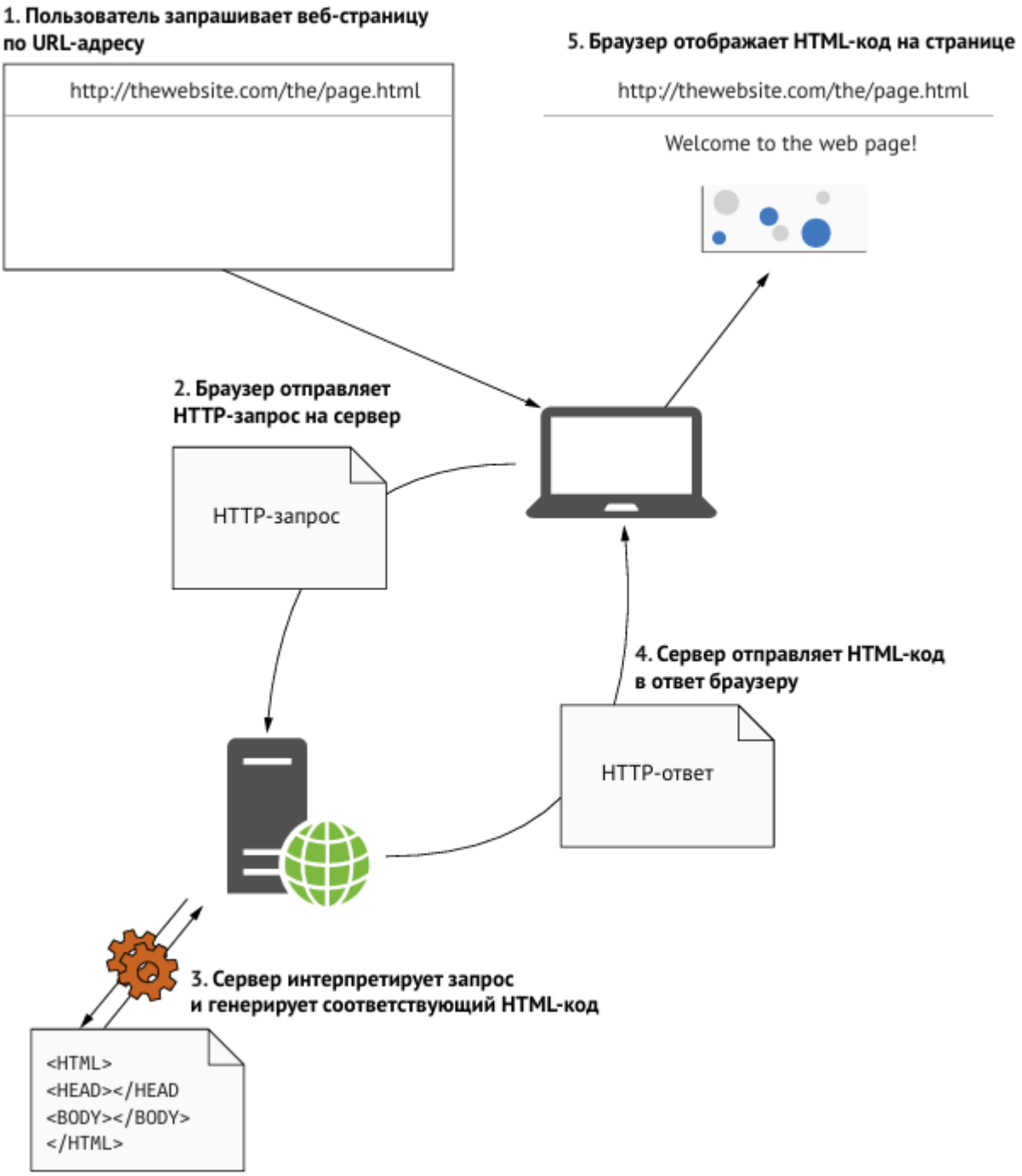
Получив запрос, Kestrel передаёт его в цепочку компонентов, каждый из которых обрабатывает запрос и либо передаёт его дальше, либо формирует ответ немедленно. В конце конвейера находится **endpoint**, который выполняет бизнес-логику и формирует итоговый ответ. Ответ проходит по конвейеру в обратном направлении, и Kestrel отправляет его клиенту.

В учебной среде Kestrel принимает запросы напрямую. В production-окружении перед Kestrel, как правило, располагается **обратный прокси-сервер** — Nginx или IIS, эту тему мы разберем подробнее в следующих лекциях.



Современные веб-архитектуры

Исторически в веб-разработке доминировал **серверный рендеринг** (Server-Side Rendering): сервер самостоятельно формировал HTML-страницу и отправлял её в браузер целиком. Именно по этому принципу работают MVC и Razor Pages. Подход прост и хорошо подходит для статичных сайтов, но плохо масштабируется при росте сложности интерфейсов.



С развитием мобильных приложений и усложнением пользовательских интерфейсов у серверного рендеринга обнаружился принципиальный недостаток: браузер, мобильное приложение и сторонние сервисы — это разные клиенты, и каждый из них не может просто «отобразить HTML». Появилась потребность в двух вещах:

- отделить фронтенд от бэкенда;
- предоставить единый API для всех клиентов — веб, мобильных и др.

Это привело к широкому распространению **SPA (Single Page Application)** — приложений, где страница загружается в браузер один раз, а все дальнейшее взаимодействие с сервером происходит через API без перезагрузки. По такому принципу работают большинство современных сайтов.

SPA и Web API

SPA — это фронтенд-приложение, написанное, как правило, на фреймворках для JavaScript: React, Vue или Angular. Оно выполняется в браузере и общается с сервером исключительно через API. Сервер при этом не знает ничего об интерфейсе — он только принимает запросы и возвращает данные.

Роль **Web API** в этой архитектуре:

- предоставляет данные в формате JSON;
- реализует бизнес-логику, авторизацию, работу с базой данных;
- обслуживает одновременно любых клиентов — браузер, мобильное приложение, desktop-клиент, партнёрские сервисы.

Рассмотрим конкретный пример. При открытии сайта в браузере происходит следующее:

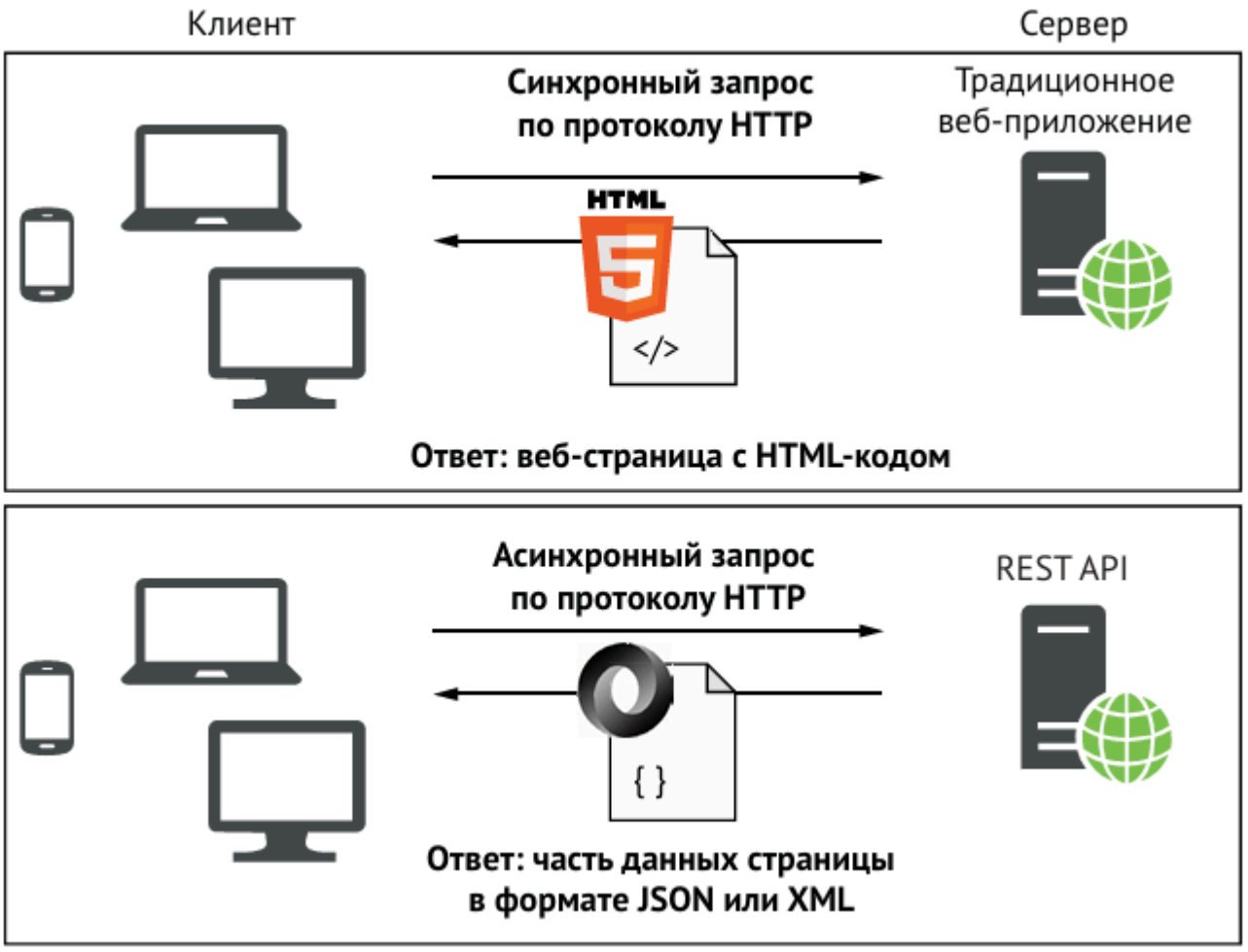
1. Браузер загружает SPA — статические файлы JS и CSS.
2. SPA отправляет запрос на сервер: `GET /api/posts`.
3. Сервер возвращает JSON со списком постов.
4. SPA самостоятельно рендерит посты в браузере.

Сервер при этом не знает, кто именно сделал запрос — браузер, мобильное приложение или сторонний клиент. Он просто возвращает данные.

Сравнение Web API и MVC / Razor Pages

	MVC / Razor Pages	**Web API**
Что возвращает	HTML	JSON
Кто рендерит UI	Сервер	Клиент
Подходит для	Блоги, лендинги	SPA, мобильные приложения, микросервисы
Зависимость от клиента	Высокая	Отсутствует

Ключевое преимущество Web API — полная независимость от способа отображения данных. Один бэкенд обслуживает любое количество клиентов, написанных на любых технологиях. Именно поэтому данный подход является стандартом современной коммерческой разработки и будет рассматриваться в этом курсе.



HTTP, REST и структура URL

Следующие понятия подробно рассматривались в лекции по сетевому программированию. Ниже приведено краткое напоминание терминологии, необходимой для работы с Web API.

HTTP и HTTPS

HTTP (HyperText Transfer Protocol) — протокол передачи данных, на котором строится всё взаимодействие в вебе. Клиент отправляет запрос, сервер возвращает ответ.

HTTPS — защищённая версия HTTP с шифрованием на основе протокола TLS. Данные, передаваемые по HTTPS, не могут быть перехвачены или подменены в процессе передачи. Все современные API работают исключительно по HTTPS.

Каждый HTTP-запрос содержит **метод**, указывающий на намерение клиента:

Метод	Действие
`GET`	Получить данные. Не изменяет состояние сервера.
`POST`	Создать новый ресурс.
`PUT`	Заменить ресурс целиком.
`PATCH`	Обновить ресурс частично.
`DELETE`	Удалить ресурс.

Сервер отвечает **кодом состояния**, сообщающим клиенту о результате операции. Коды сгруппированы по первой цифре: `2xx` — успех, `4xx` — ошибка на стороне клиента, `5xx` — ошибка на стороне сервера.

Код	Смысл	Когда возвращается
`200 OK`	Успех	GET, PUT, PATCH выполнены
`201 Created`	Ресурс создан	После успешного POST
`204 No Content`	Успех, тело ответа отсутствует	После DELETE
`400 Bad Request`	Неверный запрос	Некорректные данные от клиента
`401 Unauthorized`	Не аутентифицирован	Отсутствует или неверен токен
`403 Forbidden`	Доступ запрещён	Токен корректен, но прав недостаточно
`404 Not Found`	Ресурс не найден	Запрошенный объект не существует
`500 Internal Server Error`	Ошибка сервера	Необработанное исключение

REST API и endpoint

REST (Representational State Transfer) — архитектурный стиль построения API, основанный на нескольких принципах:

- **Ресурсо-ориентированность.** API оперирует ресурсами — пользователями, товарами, заказами. Адрес `/users/42` идентифицирует конкретного пользователя, HTTP-метод определяет операцию над ним.
- **Отсутствие состояния (Stateless).** Каждый запрос самодостаточен и содержит всю необходимую информацию. Сервер не хранит контекст между запросами.

- **Единообразие интерфейса.** Ресурсы доступны через предсказуемые URL с одинаковой логикой именования.

Endpoint (конечная точка) — конкретный адрес API, обрабатывающий запросы определённого метода. Например: `GET /users` — список пользователей, `POST /users` — создание нового, `DELETE /users/42` — удаление пользователя с идентификатором 42.

Структура URL

```
https://api.example.com:8080/users/42?sort=asc&page=1#section
  |         |              |     |         |              |
схема     домен           порт  путь    параметры       якорь
```

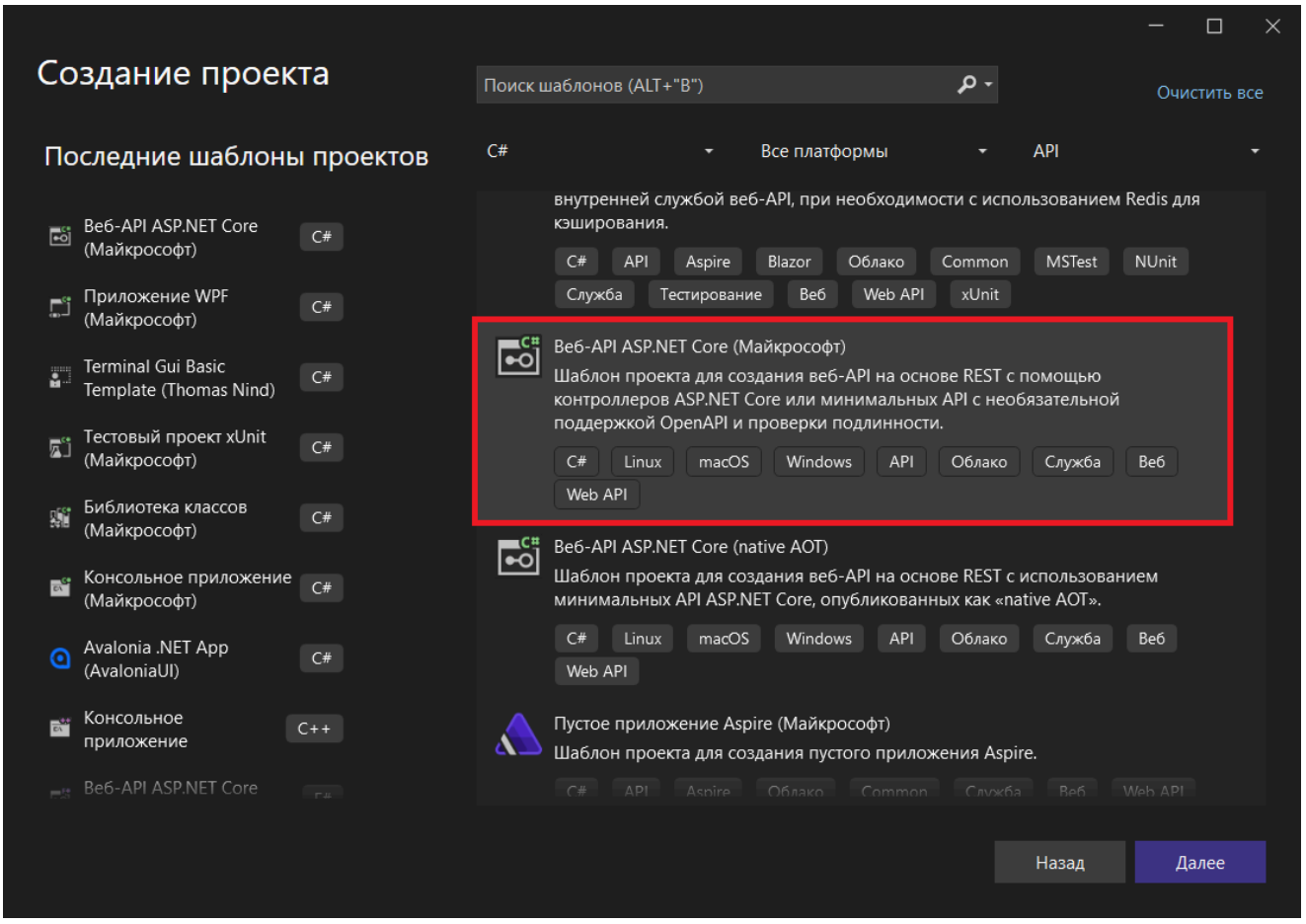
- **Схема** (`https`) — протокол передачи данных.
- **Домен** (`api.example.com`) — адрес сервера.
- **Порт** (`8080`) — сетевой порт. Если не указан, используются стандартные: `80` для HTTP, `443` для HTTPS.
- **Путь** (`/users/42`) — адрес конкретного ресурса.
- **Параметры запроса** (`sort=asc&page=1`) — дополнительные данные после символа `?`. Несколько параметров разделяются символом `&`.
- **Якорь** (`#section`) — указывает на раздел страницы. В Web API не используется.

В REST API наиболее значимы путь и параметры запроса: путь идентифицирует ресурс, параметры уточняют операцию — задают фильтрацию, сортировку, пагинацию.

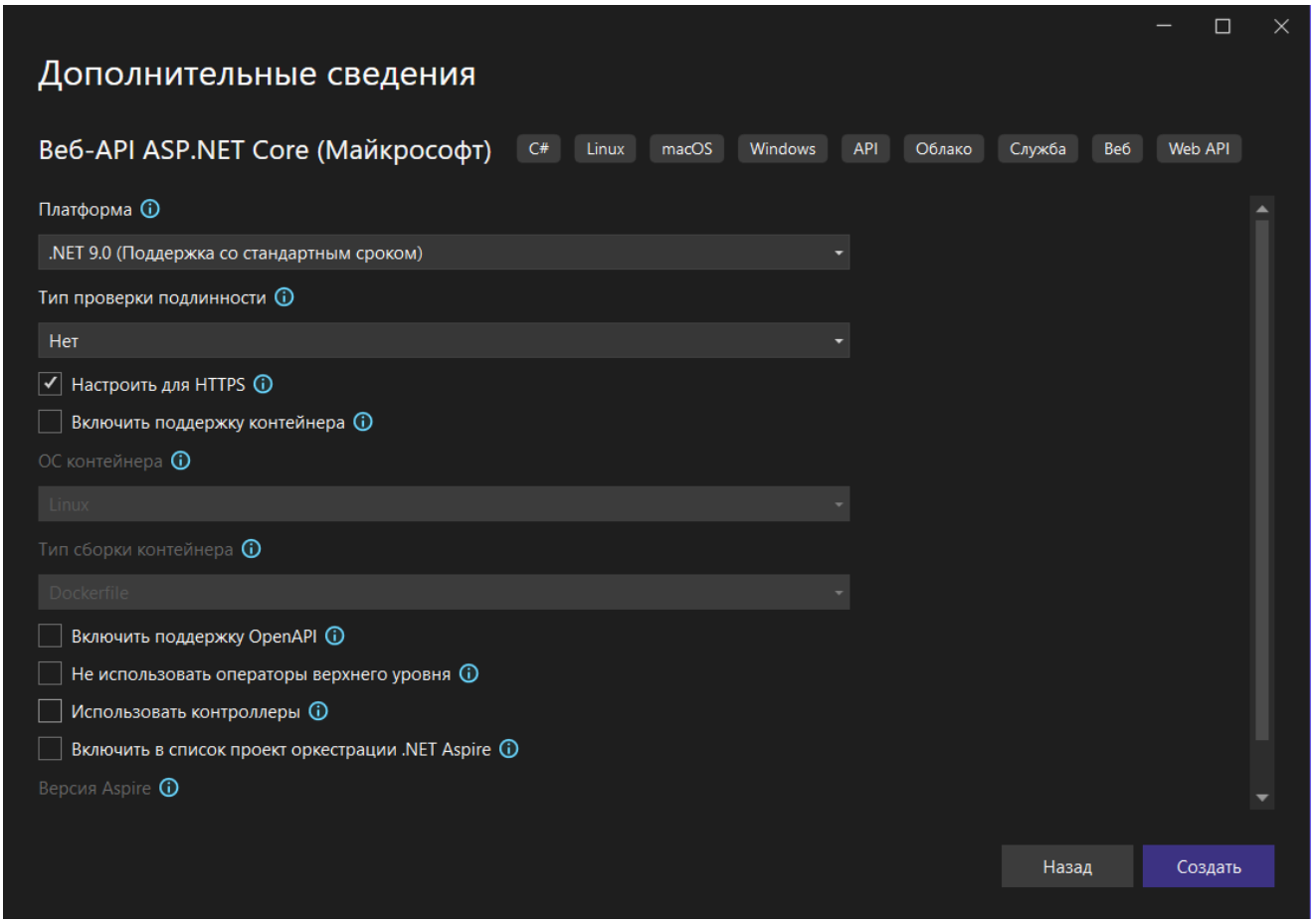
1. Первый проект: Minimal API

Создание проекта

Откройте Visual Studio и создайте новый проект. В списке шаблонов выберите **ASP.NET Core Web API**.



На следующем экране настройте параметры проекта: укажите имя и расположение. Затем на экране дополнительных параметров **снимите** все галочки, кроме **Настроить для HTTPS** — она должна остаться включённой.



После создания Visual Studio сгенерирует минимальный проект с одним тестовым эндпоинтом — прогнозом погоды.

Структура проекта

```
WeatherApi/
├── Program.cs
├── appsettings.json
├── appsettings.Development.json
├── WeatherApi.csproj
└── Properties/
    └── launchSettings.json
```

Структура намеренно минималистична. Рассмотрим назначение каждого файла.

`Program.cs` — единственный файл с кодом. Здесь находится точка входа приложения, регистрация сервисов и определение всех эндпоинтов.

`appsettings.json` — основной файл конфигурации приложения. Содержит настройки, которые применяются во всех окружениях: строки подключения к базе данных, уровни логирования, параметры сторонних сервисов.

```
{
  "Logging": {
    "LogLevel": {
```

```
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

`appsettings.Development.json` — переопределения конфигурации для режима разработки. Значения из этого файла имеют приоритет над `appsettings.json`, когда приложение запущено в окружении `Development`. Типичное использование — более подробное логирование на локальной машине или подключение к тестовой базе данных.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

`launchSettings.json` — настройки запуска для локальной разработки. Здесь определяются профили запуска, порты и переменные окружения.

```
{
  "$schema": "http://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "https://localhost:7000;http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
```

```
  }
}
```

Обратите внимание на переменную `ASPNETCORE_ENVIRONMENT`. Именно она определяет текущее окружение: когда её значение равно `Development`, приложение загружает `appsettings.Development.json` и активирует поведение, предназначенное только для разработки.

Разбор шаблонного кода

Откроем `Program.cs` и разберём его по частям.

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseHttpsRedirection();

var summaries = new[]
{
    "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot",
"Sweltering", "Scorching"
};

app.MapGet("/weatherforecast", () =>
{
    var forecast = Enumerable.Range(1, 5).Select(index =>
        new WeatherForecast
        (
            DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            Random.Shared.Next(-20, 55),
            summaries[Random.Shared.Next(summaries.Length)]
        ))
        .ToArray();
    return forecast;
});

app.Run();

internal record WeatherForecast(DateOnly Date, int TemperatureC, string?
Summary)
{
    public int TemperatureF => 32 + (int)(TemperatureC / 0.5556);
}
```

Код делится на две части, разделённые вызовом `builder.Build()`.

До `builder.Build()` — регистрация сервисов. `WebApplication.CreateBuilder(args)` создаёт строитель приложения: читает конфигурационные файлы, подготавливает DI-контейнер, инициализирует логирование. Через `builder.Services` можно зарегистрировать собственные или встроенные сервисы. В шаблоне Minimal API эта секция намеренно пуста — никаких дополнительных сервисов не требуется.

После `builder.Build()` — конфигурация конвейера запросов. `builder.Build()` фиксирует все зарегистрированные сервисы и возвращает готовый объект `WebApplication`. После этой строки регистрировать новые сервисы нельзя.

`app.UseHttpsRedirection()` — middleware, автоматически перенаправляющее HTTP-запросы на HTTPS. Запрос на `http://localhost:5000` будет перенаправлен на `https://localhost:7000`. Более подробно, что такое middleware и конвейер запросов мы разберем в следующей лекции.

`app.MapGet("/weatherforecast", ...)` — регистрация эндпоинта. Метод `MapGet` связывает HTTP-метод GET и путь `/weatherforecast` с обработчиком — лямбда-функцией, которая будет вызвана при каждом запросе. Обработчик генерирует массив из пяти прогнозов погоды со случайными данными и возвращает его. ASP.NET Core автоматически сериализует массив в JSON.

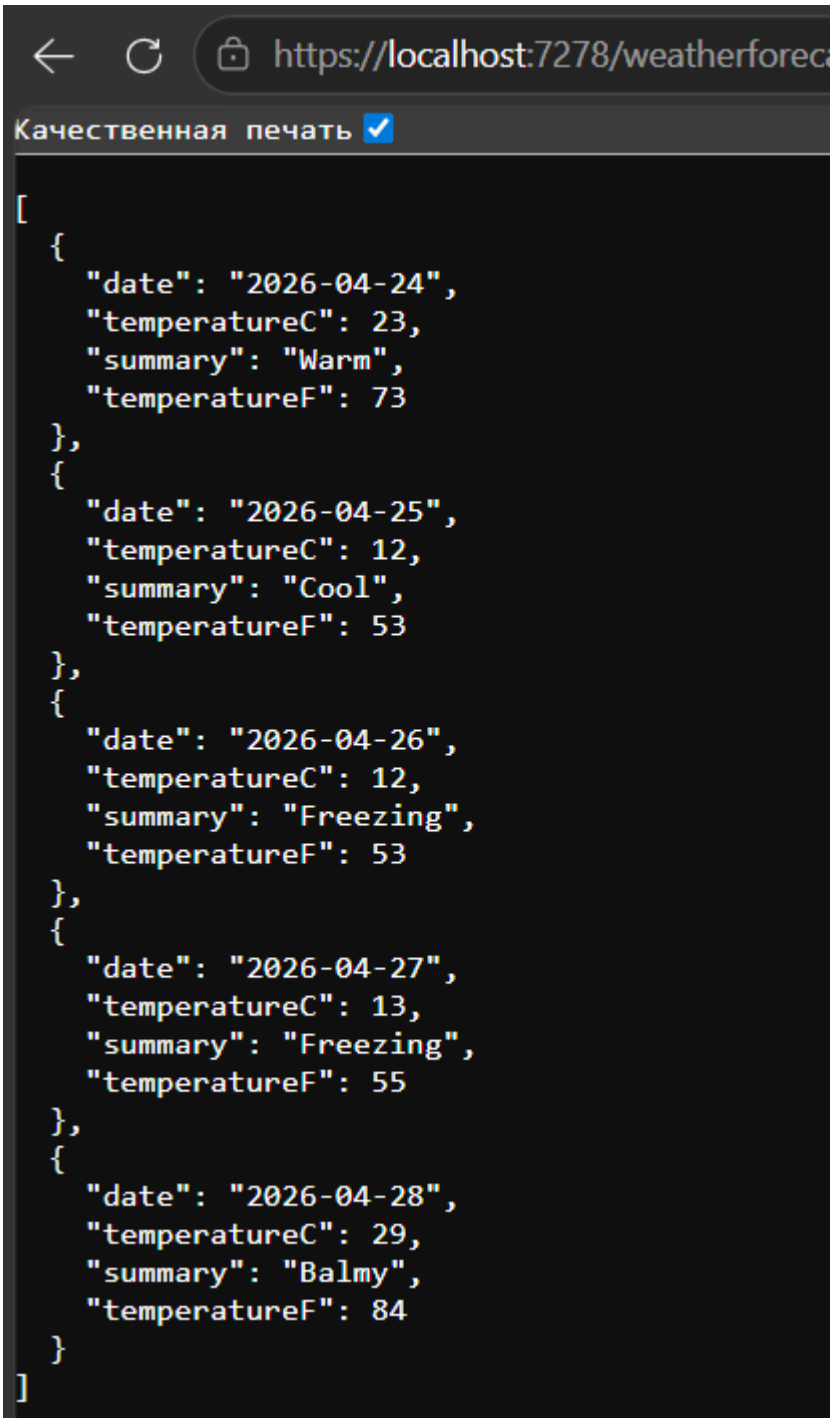
`app.Run()` — запускает HTTP-сервер и блокирует выполнение до остановки приложения.

В конце файла объявлена модель данных `WeatherForecast`. Это `record` — неизменяемый тип данных с автоматически генерируемыми конструктором и сравнением по значению. Свойство `TemperatureF` вычисляется на лету из `TemperatureC` и в конструкторе не передаётся.

После того как мы запустим сервер, откроется консоль в которую будут выводиться логи приложения.

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7278
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5167
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\andro\Desktop\test\WebApplication\WebApplication
```

Если мы откроем `https://localhost:7000/weatherforecast` (порт может отличаться). Сервер вернёт JSON с пятью прогнозами погоды.



Расширение Minimal API

Рассмотрим типичные задачи, которые возникают при реальной разработке.

Параметры в пути

Часть URL может быть параметром. Фигурные скобки в шаблоне пути обозначают переменную, которая автоматически передаётся в обработчик:

```
app.MapGet("/users/{id}", (int id) =>
{
    return Results.Ok(new { Id = id, Name = $"Пользователь {id}" });
});
```

Запрос `GET /users/42` вызовет обработчик с `id = 42`.

Параметры строки запроса

Параметры после `?` в URL передаются в обработчик автоматически — по совпадению имени:

```
app.MapGet("/users", (string? search, int page = 1) =>
{
    return Results.Ok(new { Search = search, Page = page });
});
```

Запрос `GET /users?search=Иван&page=2` вызовет обработчик с `search = "Иван"` и `page = 2`. Параметр `page` имеет значение по умолчанию и может быть опущен.

Тело запроса

Данные из тела POST-запроса десериализуются автоматически:

```
app.MapPost("/users", (UserDto user) =>
{
    return Results.Created($"/users/1", user);
});

record UserDto(string Name, string Email);
```

Возвращаемые значения и коды ответа

Класс `Results` предоставляет статические методы для формирования HTTP-ответов с явным указанием кода состояния. Рассмотрим все основные варианты.

Успешные ответы (2xx)

`Results.Ok(value)` — код `200 OK`. Используется для успешных GET, PUT, PATCH запросов. Возвращает тело ответа в формате JSON:

```
app.MapGet("/users/{id}", (int id) =>
{
    var user = new User(id, "Иван Иванов", "ivan@example.com");
    return Results.Ok(user); // 200 + JSON
});
```

`Results.Created(uri, value)` — код `201 Created`. Используется после успешного создания ресурса. Автоматически добавляет заголовок `Location` с адресом созданного ресурса:

```
app.MapPost("/users", (UserDto dto) =>
{
    var user = new User(1, dto.Name, dto.Email);
    return Results.Created($"/users/{user.Id}", user); // 201 + Location:
/users/1 + JSON
});
```

`Results.NoContent()` — код `204 No Content`. Используется для операций, которые выполнились успешно, но не возвращают данных — как правило, для DELETE и иногда PUT:

```
app.MapDelete("/users/{id}", (int id) =>
{
    // удаление...
    return Results.NoContent(); // 204, тело ответа отсутствует
});
```

Ошибки клиента (4xx)

`Results.BadRequest(error)` — код `400 Bad Request`. Используется, когда клиент передал некорректные данные: нарушены бизнес-правила, отсутствуют обязательные поля, неверный формат:

```
app.MapPost("/users", (UserDto dto) =>
{
    if (string.IsNullOrWhiteSpace(dto.Name))
        return Results.BadRequest("Имя пользователя обязательно."); // 400 +
текст ошибки

    if (dto.Name.Length > 100)
        return Results.BadRequest(new { Field = "Name", Error = "Имя не
может превышать 100 символов." }); // 400 + JSON

    // создание...
    return Results.Created($"/users/1", dto);
});
```

В `BadRequest` можно передать как строку, так и произвольный объект — он будет сериализован в JSON. Второй вариант предпочтительнее, если нужно сообщить клиенту о нескольких ошибках сразу.

`Results.NotFound()` — код `404 Not Found`. Используется, когда запрошенный ресурс не существует:

```
app.MapGet("/users/{id}", (int id) =>
{
```

```
    var user = users.FirstOrDefault(u => u.Id == id);

    if (user is null)
        return Results.NotFound(); // 404, тело отсутствует

    return Results.Ok(user);
});
```

Можно передать поясняющее сообщение:

```
return Results.NotFound($"Пользователь с id={id} не найден."); // 404 +
текст
```

Все остальные коды

`Results.StatusCode(code)` — произвольный код ответа. Используется для случаев, которые не покрыты именованными методами:

```
app.MapGet("/external-data", async () =>
{
    try
    {
        // обращение к внешнему сервису...
        return Results.Ok(data);
    }
    catch (HttpRequestException)
    {
        return Results.StatusCode(503); // 503 Service Unavailable
    }
});
```

Полный пример: CRUD для списка пользователей

Соберём все приёмы вместе и реализуем полноценный CRUD. Данные хранятся в памяти — фокус на синтаксисе Minimal API, не на базе данных.

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseHttpsRedirection();

// Хранилище в памяти
// В реальном приложении — база данных через репозиторий
var users  = new List<User>();
var nextId = 1;

// GET /users — получить всех пользователей
```

```
app.MapGet("/users", () =>
    Results.Ok(users));

// GET /users/{id} — получить одного пользователя
app.MapGet("/users/{id}", (int id) =>
{
    var user = users.FirstOrDefault(u => u.Id == id);
    return user is null
        ? Results.NotFound($"Пользователь с id={id} не найден")
        : Results.Ok(user);
});

// POST /users — создать пользователя
app.MapPost("/users", (UserDto dto) =>
{
    if (string.IsNullOrWhiteSpace(dto.Name))
        return Results.BadRequest("Имя пользователя обязательно.");

    var user = new User(nextId++, dto.Name.Trim(), dto.Email);
    users.Add(user);

    // 201 Created + заголовок Location: /users/1
    return Results.Created($"/users/{user.Id}", user);
});

// PUT /users/{id} — обновить пользователя
app.MapPut("/users/{id}", (int id, UserDto dto) =>
{
    var index = users.FindIndex(u => u.Id == id);
    if (index == -1)
        return Results.NotFound($"Пользователь с id={id} не найден");

    if (string.IsNullOrWhiteSpace(dto.Name))
        return Results.BadRequest("Имя пользователя обязательно.");

    // record неизменяем — создаём новый экземпляр с обновлёнными полями
    users[index] = users[index] with { Name = dto.Name.Trim(), Email =
dto.Email };
    return Results.Ok(users[index]);
});

// DELETE /users/{id} — удалить пользователя
app.MapDelete("/users/{id}", (int id) =>
{
    var user = users.FirstOrDefault(u => u.Id == id);
    if (user is null)
        return Results.NotFound($"Пользователь с id={id} не найден");

    users.Remove(user);
    return Results.NoContent(); // 204 — успех, возвращать нечего
```

```
});

app.Run();

// Модели
record User(int Id, string Name, string Email);
record UserDto(string Name, string Email);
```

Группировка эндпоинтов

Когда эндпоинтов становится больше, префикс маршрута начинает дублироваться в каждой строке. Метод `MapGroup` позволяет указать общий префикс один раз:

```
// Без группировки — префикс /users повторяется везде
app.MapGet("/users", ...);
app.MapGet("/users/{id}", ...);
app.MapPost("/users", ...);
app.MapPut("/users/{id}", ...);
app.MapDelete("/users/{id}", ...);

// С группировкой — префикс указывается один раз
var usersGroup = app.MapGroup("/users");

usersGroup.MapGet("",        () => ...);            // GET    /users
usersGroup.MapGet("/{id}",   (int id) => ...);      // GET    /users/{id}
usersGroup.MapPost("",       (...) => ...);         // POST   /users
usersGroup.MapPut("/{id}",   (...) => ...);         // PUT    /users/{id}
usersGroup.MapDelete("/{id}", (...) => ...);        // DELETE /users/{id}
```

Вынос обработчиков из Program.cs

По мере роста проекта лямбды в `Program.cs` разрастаются и файл становится трудночитаемым. Обработчики можно вынести в отдельный статический класс:

```
// UserHandlers.cs
public static class UserHandlers
{
    public static IResult GetAll(List<User> users)
        => Results.Ok(users);

    public static IResult GetById(int id, List<User> users)
    {
        var user = users.FirstOrDefault(u => u.Id == id);
        return user is null
            ? Results.NotFound($"Пользователь с id={id} не найден")
            : Results.Ok(user);
    }
```

```
    public static IResult Create(UserDto dto, List<User> users, ref int
nextId)
    {
        if (string.IsNullOrWhiteSpace(dto.Name))
            return Results.BadRequest("Имя пользователя обязательно.");

        var user = new User(nextId++, dto.Name.Trim(), dto.Email);
        users.Add(user);
        return Results.Created($"/users/{user.Id}", user);
    }

    public static IResult Update(int id, UserDto dto, List<User> users)
    {
        var index = users.FindIndex(u => u.Id == id);
        if (index == -1)
            return Results.NotFound($"Пользователь с id={id} не найден");

        if (string.IsNullOrWhiteSpace(dto.Name))
            return Results.BadRequest("Имя пользователя обязательно.");

        users[index] = users[index] with { Name = dto.Name.Trim(), Email =
dto.Email };
        return Results.Ok(users[index]);
    }

    public static IResult Delete(int id, List<User> users)
    {
        var user = users.FirstOrDefault(u => u.Id == id);
        if (user is null)
            return Results.NotFound($"Пользователь с id={id} не найден");

        users.Remove(user);
        return Results.NoContent();
    }
}
```

`Program.cs` при этом становится компактным и читаемым:

```
var usersGroup = app.MapGroup("/users");

usersGroup.MapGet("",          UserHandlers.GetAll);
usersGroup.MapGet("/{id}",     UserHandlers.GetById);
usersGroup.MapPost("",         UserHandlers.Create);
usersGroup.MapPut("/{id}",     UserHandlers.Update);
usersGroup.MapDelete("/{id}", UserHandlers.Delete);
```

Ограничения Minimal API

Вынос обработчиков в статические классы решает проблему читаемости, однако архитектура остаётся плоской. При дальнейшем росте проекта возникают новые сложности.

Статические методы плохо сочетаются с внедрением зависимостей: передавать репозитории, сервисы и другие зависимости в каждый метод вручную неудобно. Применить единую логику — например, авторизацию или валидацию — к группе связанных эндпоинтов сложнее, чем хотелось бы. Наконец, при большом количестве ресурсов (`/users` , `/orders` , `/products` , ...) даже компактный `Program.cs` с регистрацией всех групп становится громоздким.

Для решения этих проблем в ASP.NET Core предусмотрен более структурированный подход — **контроллеры**.

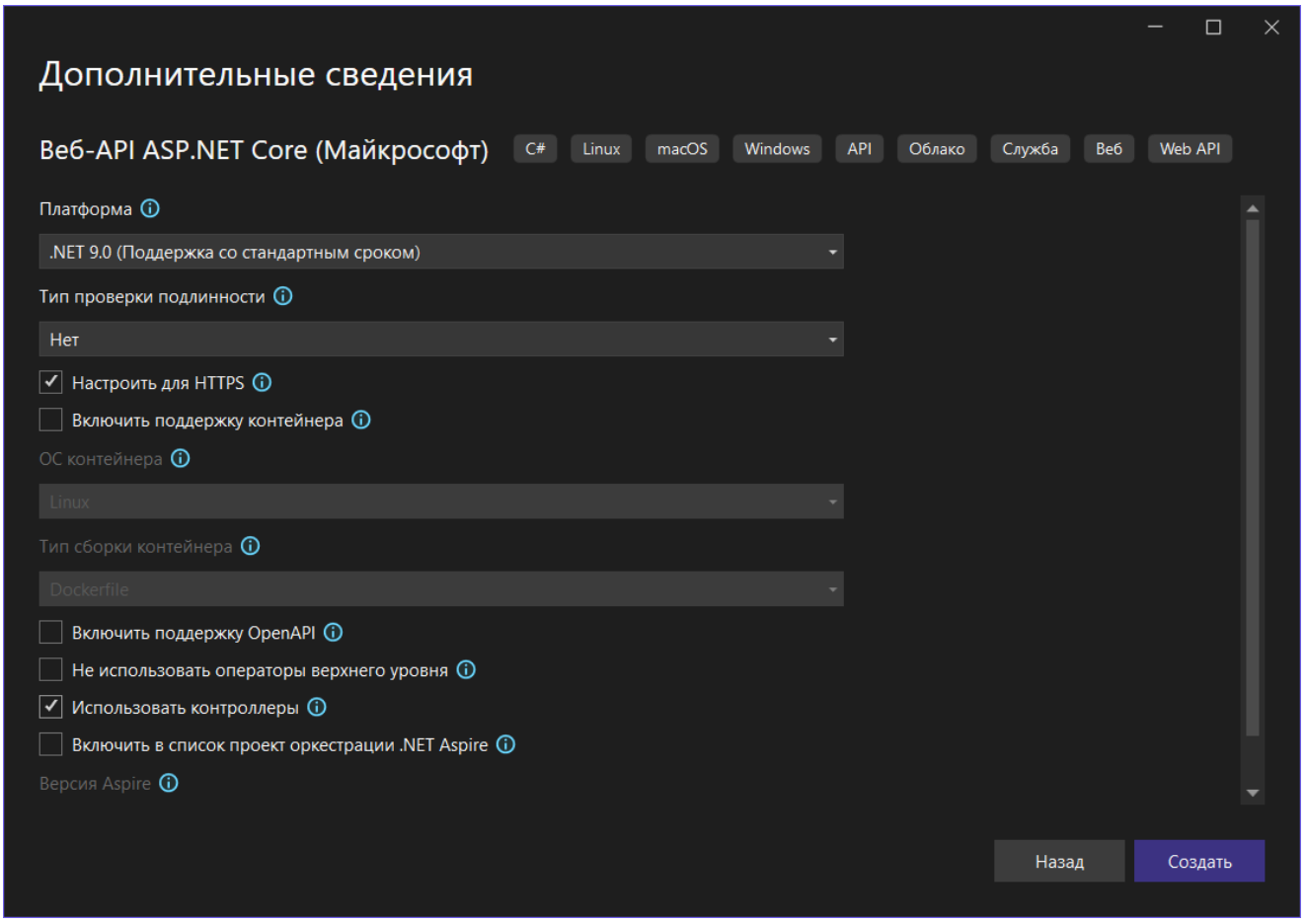
2. Контроллеры

Контроллер — это класс, который группирует эндпоинты, логически связанные с одним ресурсом. Все операции с пользователями находятся в `UsersController`, все операции с заказами — в `OrdersController`. Каждый публичный метод контроллера, помеченный атрибутом HTTP-метода, становится отдельным эндпоинтом.

В отличие от статических обработчиков Minimal API, контроллер — это обычный класс, полностью интегрированный с DI-контейнером. Зависимости (репозитории, сервисы, логгеры) передаются через конструктор, ASP.NET Core создаёт экземпляр контроллера и разрешает все зависимости автоматически при каждом запросе.

Создание проекта с контроллерами

Создайте новый проект ASP.NET Core Web API в Visual Studio. На экране дополнительных параметров поставьте галочку напротив **Использовать контроллеры**.



Visual Studio создаст проект со следующей структурой:

```
WeatherApi/
├── Controllers/
│   └── WeatherForecastController.cs
├── Program.cs
├── WeatherForecast.cs
```

```
├── appsettings.json
├── appsettings.Development.json
├── WeatherApi.csproj
└── Properties/
    └── launchSettings.json
```

Появились два новых элемента: папка `Controllers` и отдельный файл модели `WeatherForecast.cs`. Рассмотрим все файлы по порядку.

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();

var app = builder.Build();

// Configure the HTTP request pipeline.
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

По сравнению с Minimal API появились три новых строки. Разберём каждую.

`builder.Services.AddControllers()` регистрирует в DI-контейнере все сервисы, необходимые для работы контроллеров: систему маршрутизации, сериализацию JSON, валидацию моделей, фильтры и многое другое. Без этого вызова ASP.NET Core не будет знать о существовании контроллеров в проекте.

Помимо этого, `AddControllers` сканирует сборку и регистрирует в DI-контейнере все классы-контроллеры. Это означает, что при входящем запросе фреймворк самостоятельно создаст нужный контроллер, разрешит все его зависимости через DI и уничтожит экземпляр после обработки запроса. Явно создавать контроллер через `new` не нужно и не следует.

`app.UseHttpsRedirection()` — middleware, перенаправляющее HTTP-запросы на HTTPS. Работает так же, как в Minimal API.

`app.UseAuthorization()` подключает middleware проверки прав доступа, подробнее про то для чего он нужен, мы разберем в следующих лекциях, пока он для нас бесполезен.

`app.MapControllers()` сканирует все зарегистрированные контроллеры и подключает их маршруты к конвейеру запросов. Без этой строки контроллеры будут

зарегистрированы в DI-контейнере, но запросы до них не дойдут.

WeatherForecast.cs

```
namespace WebApplication;

public class WeatherForecast
{
    public DateOnly Date { get; set; }
    public int TemperatureC { get; set; }
    public int TemperatureF => 32 + (int)(TemperatureC / 0.5556);
    public string? Summary { get; set; }
}
```

В проектах с контроллерами модели принято хранить отдельно: в папке `Models` или рядом с контроллером. Это улучшает навигацию по проекту по мере его роста.

WeatherForecastController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace WebApplication.Controllers;

[ApiController]
[Route("[controller]")]
public class WeatherForecastController : ControllerBase
{
    private static readonly string[] Summaries = new[]
    {
        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm",
        "Balmy", "Hot", "Sweltering", "Scorching"
    };

    private readonly ILogger<WeatherForecastController> _logger;

    public WeatherForecastController(ILogger<WeatherForecastController>
logger)
    {
        _logger = logger;
    }

    [HttpGet]
    public IEnumerable<WeatherForecast> Get()
    {
        return Enumerable.Range(1, 5).Select(index => new WeatherForecast
        {
            Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            TemperatureC = Random.Shared.Next(-20, 55),
```

```
            Summary = Summaries[Random.Shared.Next(Summaries.Length)]
        })
        .ToArray();
    }
}
```

Разберём каждую составляющую.

Базовый класс и атрибуты

`ControllerBase` — базовый класс для API-контроллеров. Предоставляет вспомогательные методы для формирования ответов: `Ok()`, `NotFound()`, `BadRequest()`, `Created()` и другие, а также доступ к объектам `Request`, `Response`, `User` и `HttpContext`.

`[ApiController]` — атрибут, активирующий набор соглашений, упрощающих разработку API:

- автоматическая валидация модели: если входящие данные не прошли валидацию, контроллер вернёт `400 Bad Request` без дополнительного кода;
- автоматическое определение источника параметров: фреймворк сам понимает, что сложный объект нужно читать из тела запроса, а простые типы — из пути или строки запроса;
- улучшенные сообщения об ошибках в стандартном формате `ProblemDetails`.

`[Route("[controller]")]` задаёт шаблон маршрута для всего контроллера. Placeholder `[controller]` автоматически заменяется на имя класса без суффикса `Controller`. Для `WeatherForecastController` маршрут будет `/WeatherForecast`. Распространённая практика — использовать префикс `api`: `[Route("api/[controller]")]`, тогда маршрут станет `/api/WeatherForecast`.

Внедрение зависимостей через конструктор

```
private readonly ILogger<WeatherForecastController> _logger;

public WeatherForecastController(ILogger<WeatherForecastController> logger)
{
    _logger = logger;
}
```

Это стандартный DI, что использовался в лекциях по архитектуре. `ILogger` зарегистрирован в DI-контейнере фреймворком автоматически, поэтому его не нужно регистрировать вручную в `Program.cs`. При создании контроллера ASP.NET Core увидит, что конструктор требует `ILogger<WeatherForecastController>`, найдёт его в контейнере и передаст автоматически.

По тому же принципу передаются любые зависимости: репозитории, сервисы, конфигурация.

Метод-обработчик

```
[HttpGet]
public IEnumerable<WeatherForecast> Get()
{
    ...
}
```

`[HttpGet]` указывает, что метод обрабатывает GET-запросы. Аналогичные атрибуты существуют для всех HTTP-методов: `[HttpPost]`, `[HttpPut]`, `[HttpPatch]`, `[HttpDelete]`.

Метод возвращает `IEnumerable<WeatherForecast>` — ASP.NET Core автоматически сериализует результат в JSON и вернёт ответ с кодом `200 OK`.

Получение параметров

Контроллеры используют атрибуты для явного указания источника параметров. Это делает код более читаемым и устраняет неоднозначность.

Параметр из пути — `[FromRoute]`

```
[HttpGet("{id}")]
public ActionResult<User> GetById([FromRoute] int id)
{
    var user = _users.FirstOrDefault(u => u.Id == id);
    return user is null ? NotFound() : Ok(user);
}
```

Шаблон `"{id}"` в атрибуте `[HttpGet]` дополняет маршрут контроллера. Если контроллер имеет маршрут `api/users`, то полный маршрут метода будет `GET api/users/{id}`.

Параметр из строки запроса — `[FromQuery]`

```
[HttpGet]
public ActionResult<IEnumerable<User>> GetAll([FromQuery] string? search,
                                              [FromQuery] int page = 1)
{
    var result = string.IsNullOrEmpty(search)
        ? _users
        : _users.Where(u => u.Name.Contains(search));
```

```
    return Ok(result.Skip((page - 1) * 10).Take(10));
}
```

Запрос `GET api/users?search=Иван&page=2` передаст `search = "Иван"` и `page = 2`.

Параметр из тела запроса — `[FromBody]`

```
[HttpPost]
public ActionResult<User> Create([FromBody] UserDto dto)
{
    var user = new User(_nextId++, dto.Name, dto.Email);
    _users.Add(user);
    return CreatedAtAction(nameof(GetById), new { id = user.Id }, user);
}
```

`[FromBody]` читает тело запроса и десериализует JSON в объект `UserDto`. При наличии атрибута `[ApiController]` указывать `[FromBody]` явно не обязательно — фреймворк определяет источник автоматически для сложных типов. Тем не менее явное указание улучшает читаемость кода.

`CreatedAtAction` формирует ответ `201 Created` и автоматически добавляет заголовок `Location` с адресом созданного ресурса, используя маршрут указанного метода.

Возвращаемые значения

Методы контроллера могут возвращать несколько типов.

`IActionResult` — базовый интерфейс для всех результатов. Используется, когда метод может вернуть разные типы данных в зависимости от ситуации:

```
[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    var user = _users.FirstOrDefault(u => u.Id == id);

    if (user is null)
        return NotFound();       // 404

    return Ok(user);             // 200 + JSON
}
```

`ActionResult<T>` — обобщённая версия, позволяющая явно указать тип успешного ответа. Это улучшает документацию и подсказки компилятора:

```
[HttpGet("{id}")]
public ActionResult<User> GetById(int id)
{
```

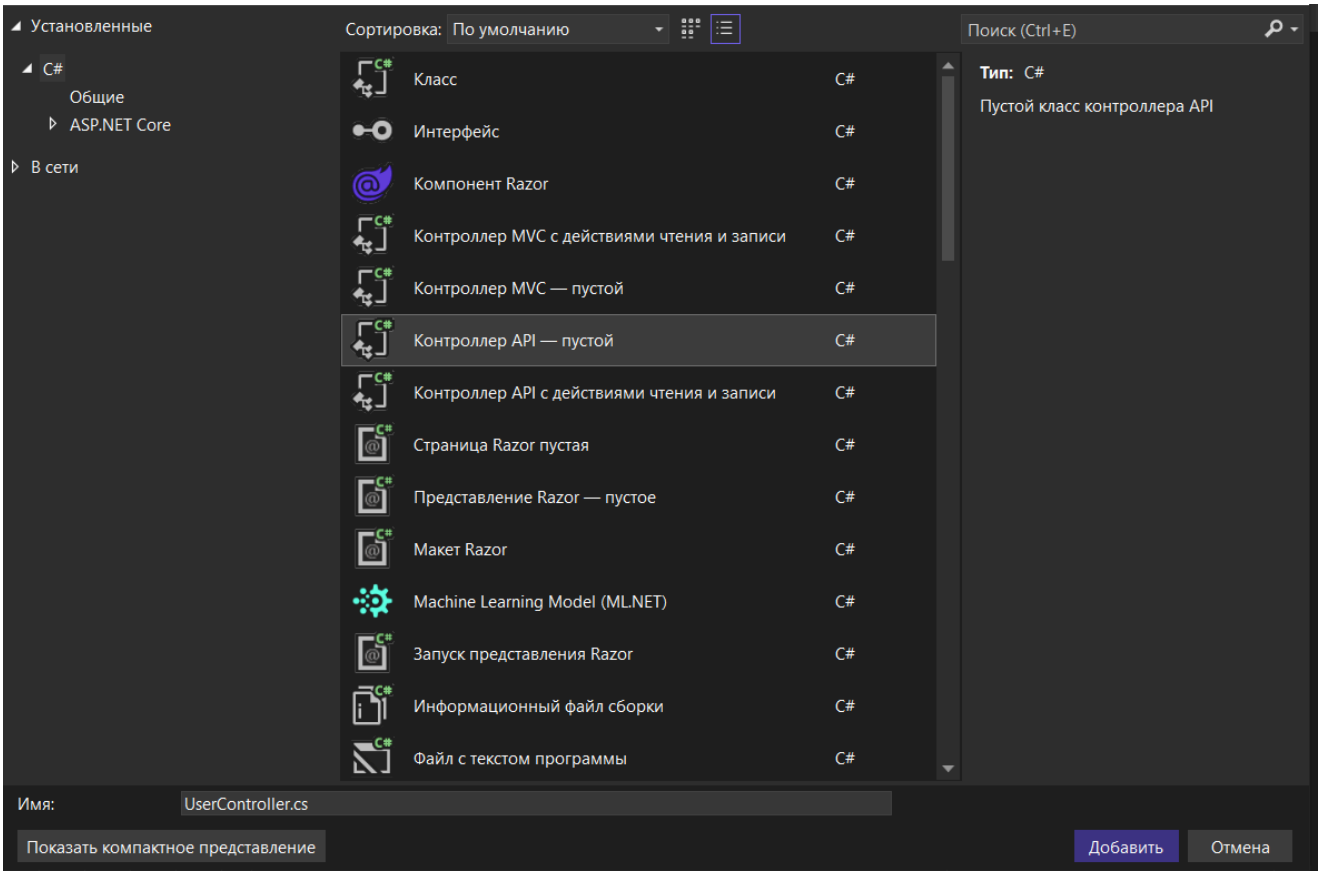
```
    var user = _users.FirstOrDefault(u => u.Id == id);
    return user is null ? NotFound() : Ok(user);
}
```

Вспомогательные методы, унаследованные от `ControllerBase`:

Метод	Код	Описание
`Ok(value)`	200	Успешный ответ с телом
`Created(uri, value)`	201	Ресурс создан
`CreatedAtAction(...)`	201	Ресурс создан, Location из маршрута
`NoContent()`	204	Успех без тела ответа
`BadRequest(...)`	400	Ошибка в данных запроса
`NotFound()`	404	Ресурс не найден
`StatusCode(code, value)`	любой	Произвольный код ответа

CRUD на контроллерах

Для того чтобы создать новый контроллер, нужно нажать ПКМ на папку **Controllers** -> **Добавить** -> **Создать элемент**, После чего выбираем **Контроллер API — пустой**.



Реализуем тот же CRUD для пользователей, что был написан на Minimal API, — теперь в виде контроллера.

```
using Microsoft.AspNetCore.Mvc;

namespace WebApplication.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class UsersController : ControllerBase
    {
        // Хранилище в памяти — в реальном приложении заменяется
репозиторием
        private static readonly List<User> _users = new();
        private static int _nextId = 1;

        // GET api/users
        // GET api/users?search=Иван&page=2
        [HttpGet]
        public ActionResult<IEnumerable<User>> GetAll([FromQuery] string?
search,
                                                      [FromQuery] int page =
1)
        {
            var result = string.IsNullOrEmpty(search)
                ? _users
                : _users.Where(u => u.Name.Contains(search)).ToList();

            return Ok(result);
        }

        // GET api/users/42
        [HttpGet("{id}")]
        public ActionResult<User> GetById([FromRoute] int id)
        {
            var user = _users.FirstOrDefault(u => u.Id == id);
            return user is null ? NotFound() : Ok(user);
        }

        // POST api/users
        [HttpPost]
        public ActionResult<User> Create([FromBody] UserDto dto)
        {
            if (string.IsNullOrWhiteSpace(dto.Name))
                return BadRequest("Имя пользователя обязательно.");

            var user = new User(_nextId++, dto.Name.Trim(), dto.Email);
            _users.Add(user);

            return CreatedAtAction(nameof(GetById), new { id = user.Id },
user);
        }
```

```
        // PUT api/users/42
        [HttpPut("{id}")]
        public ActionResult<User> Update([FromRoute] int id, [FromBody]
UserDto dto)
        {
            var index = _users.FindIndex(u => u.Id == id);
            if (index == -1)
                return NotFound();

            if (string.IsNullOrWhiteSpace(dto.Name))
                return BadRequest("Имя пользователя обязательно.");

            _users[index] = new User(id, dto.Name.Trim(), dto.Email);
            return Ok(_users[index]);
        }

        // DELETE api/users/42
        [HttpDelete("{id}")]
        public IActionResult Delete([FromRoute] int id)
        {
            var user = _users.FirstOrDefault(u => u.Id == id);
            if (user is null)
                return NotFound();

            _users.Remove(user);
            return NoContent();
        }
    }
}

// Модели

public record User(int Id, string Name, string Email);
public record UserDto(string Name, string Email);
```

Minimal API vs Controllers

Оба подхода решают одну задачу и могут сосуществовать в одном проекте. Выбор определяется характером задачи.

Minimal API предпочтительнее, когда:

- проект небольшой: несколько эндпоинтов, один разработчик;
- требуется максимальная простота и минимум шаблонного кода;
- разрабатывается микросервис с узкой ответственностью.

Контроллеры предпочтительнее, когда:

- проект содержит несколько ресурсов с полным набором операций;
- важна единообразная структура, понятная всей команде;
- требуются фильтры, расширенная валидация или сложная авторизация.

На практике в большинстве коммерческих проектов используются контроллеры — именно этот подход будет применяться в дальнейших лекциях курса.

3. Swagger

Что такое Swagger и OpenAPI

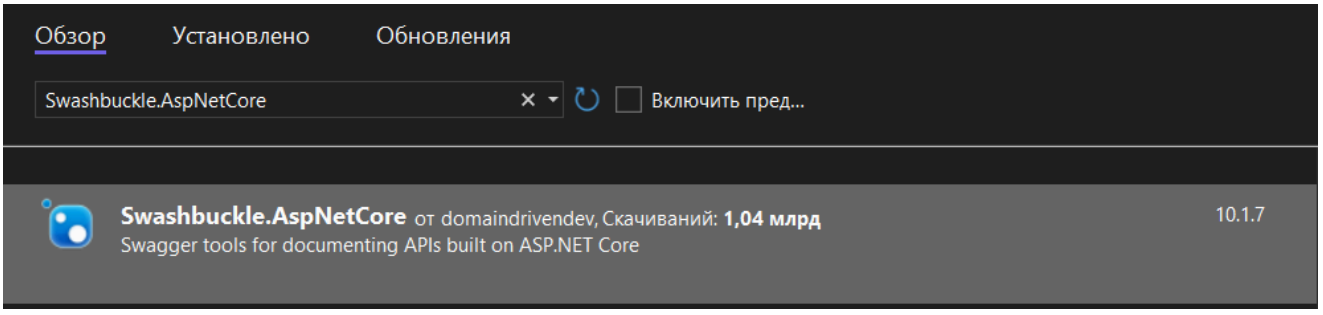
OpenAPI — стандарт описания REST API. Это машиночитаемый документ в формате JSON или YAML, который содержит полное описание API: все эндпоинты, параметры, форматы запросов и ответов, коды состояний. По такому документу можно автоматически сгенерировать клиентский код, тесты или документацию.

Swagger — набор инструментов для работы с OpenAPI. В контексте ASP.NET Core нас интересуют два компонента: **Swashbuckle** генерирует OpenAPI-документ автоматически из кода контроллеров, **Swagger UI** отображает этот документ как интерактивную веб-страницу, на которой можно тестировать эндпоинты прямо в браузере.

Подключение

Установка пакета

Swagger не входит в стандартную библиотеку ASP.NET Core и устанавливается отдельно. В Visual Studio откройте **Управление пакетами NuGet** и установите пакет `Swashbuckle.AspNetCore`.



Альтернативно — через консоль диспетчера пакетов:

```
dotnet add package Swashbuckle.AspNetCore
```

Изменения в Program.cs

Добавьте регистрацию сервисов до `builder.Build()` и подключение middleware после:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Регистрация сервисов Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();

// Подключение Swagger — только в режиме разработки
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

`AddEndpointsApiExplorer()` регистрирует сервис, который сканирует все эндпоинты приложения и предоставляет метаданные о них. Swashbuckle использует эти метаданные для построения OpenAPI-документа.

`AddSwaggerGen()` регистрирует генератор OpenAPI-документа. При запуске приложения он обходит все контроллеры и строит описание API.

`app.UseSwagger()` добавляет эндпоинт `GET /swagger/v1/swagger.json`, возвращающий сгенерированный OpenAPI-документ в формате JSON.

`app.UseSwaggerUI()` добавляет эндпоинт `GET /swagger`, отдающий интерактивный веб-интерфейс, который читает документ из `/swagger/v1/swagger.json` и отображает его в виде удобной страницы.

Оба вызова обёрнуты в `if (app.Environment.IsDevelopment())`. В production Swagger нежелателен: документация раскрывает полную структуру API и может быть использована злоумышленниками.

Настройка автоматического открытия в браузере

По умолчанию при запуске проекта браузер открывается на корневом URL. Настроим `launchSettings.json` так, чтобы Swagger открывался автоматически.

Откройте `Properties/launchSettings.json` и добавьте `"launchUrl": "swagger"` в нужный профиль:

```
{
  "$schema": "http://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
```

```
      "launchUrl": "swagger",
      "applicationUrl": "http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:7000;http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Теперь при нажатии кнопки запуска в Visual Studio браузер сразу откроется на странице Swagger UI.

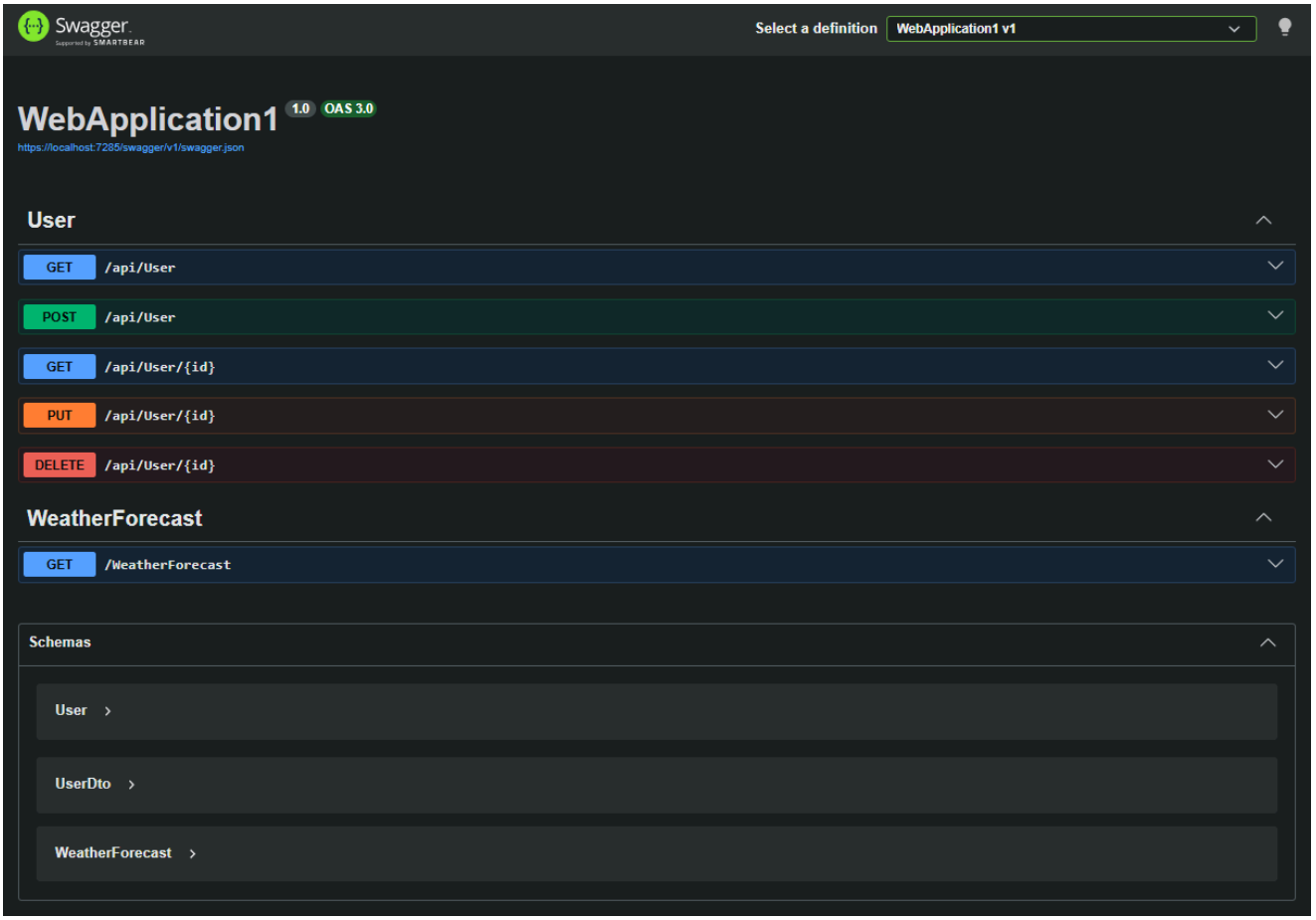
Если удобнее, чтобы Swagger открывался на корневом URL `/` вместо `/swagger`, настройте это в `Program.cs`:

```
app.UseSwaggerUI(options =>
{
    options.RoutePrefix = string.Empty; // Swagger UI на
https://localhost:7000/
});
```

В этом случае значение `launchUrl` в `launchSettings.json` можно убрать или оставить пустой строкой.

Возможности Swagger UI

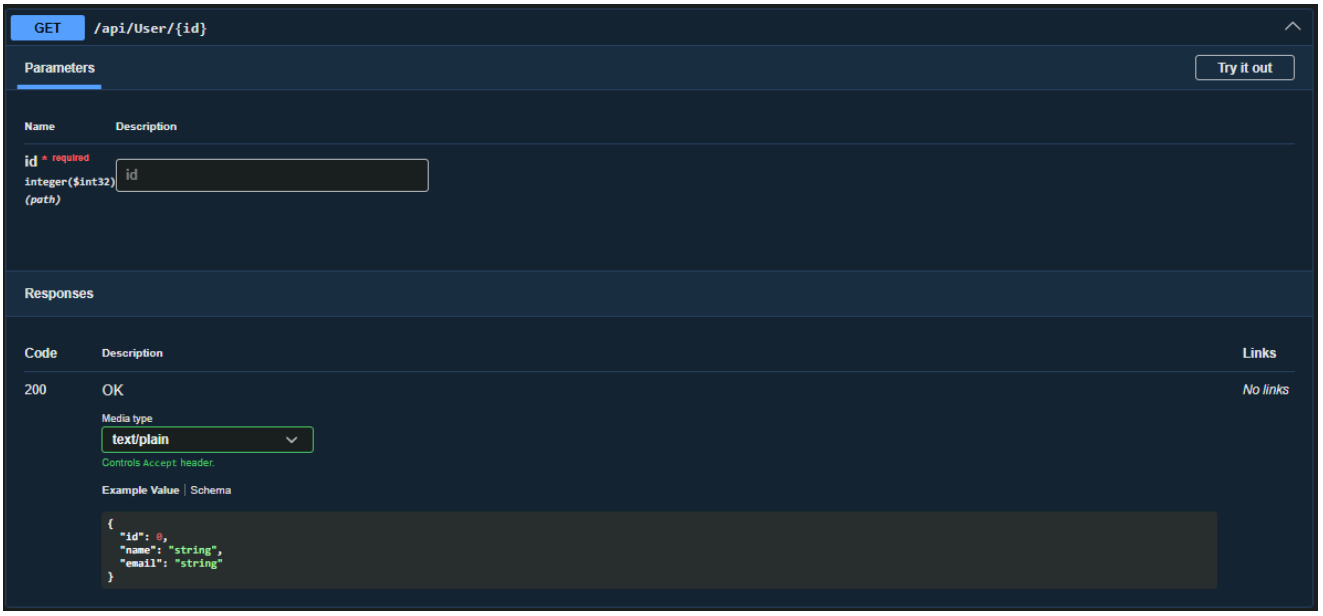
Запустите проект и откройте страницу Swagger UI.



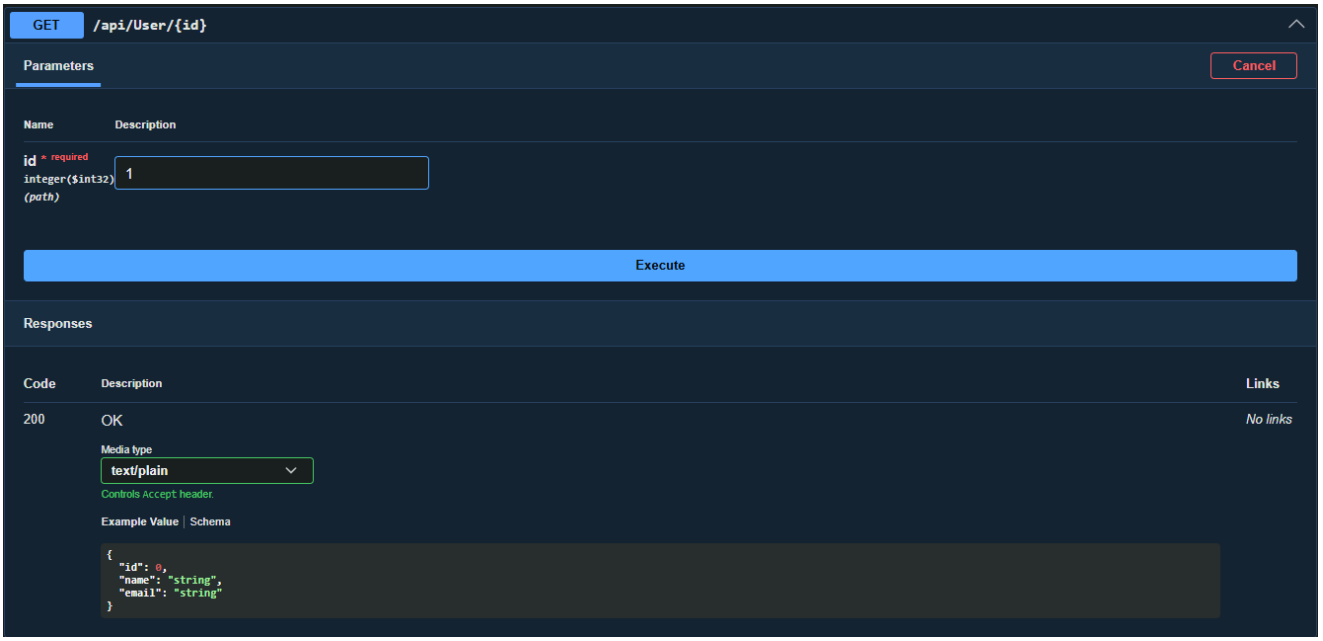
Интерфейс отображает все эндпоинты контроллера, сгруппированные по имени контроллера. Для каждого эндпоинта показаны HTTP-метод, маршрут и краткое описание.

Тестирование эндпоинта

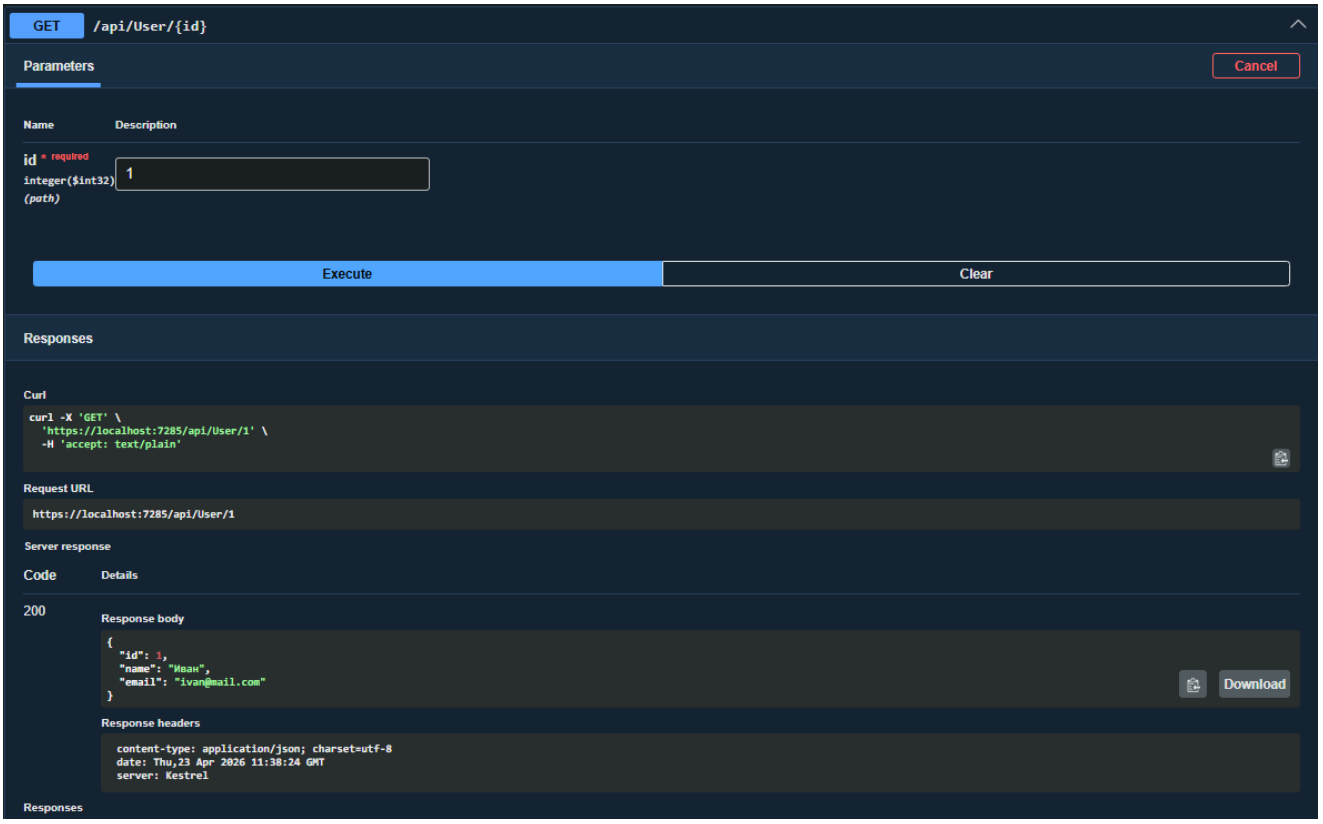
Нажмите на любой эндпоинт — раскроется детальная карточка.



Нажмите **Try it out**, заполните параметры и нажмите **Execute**.

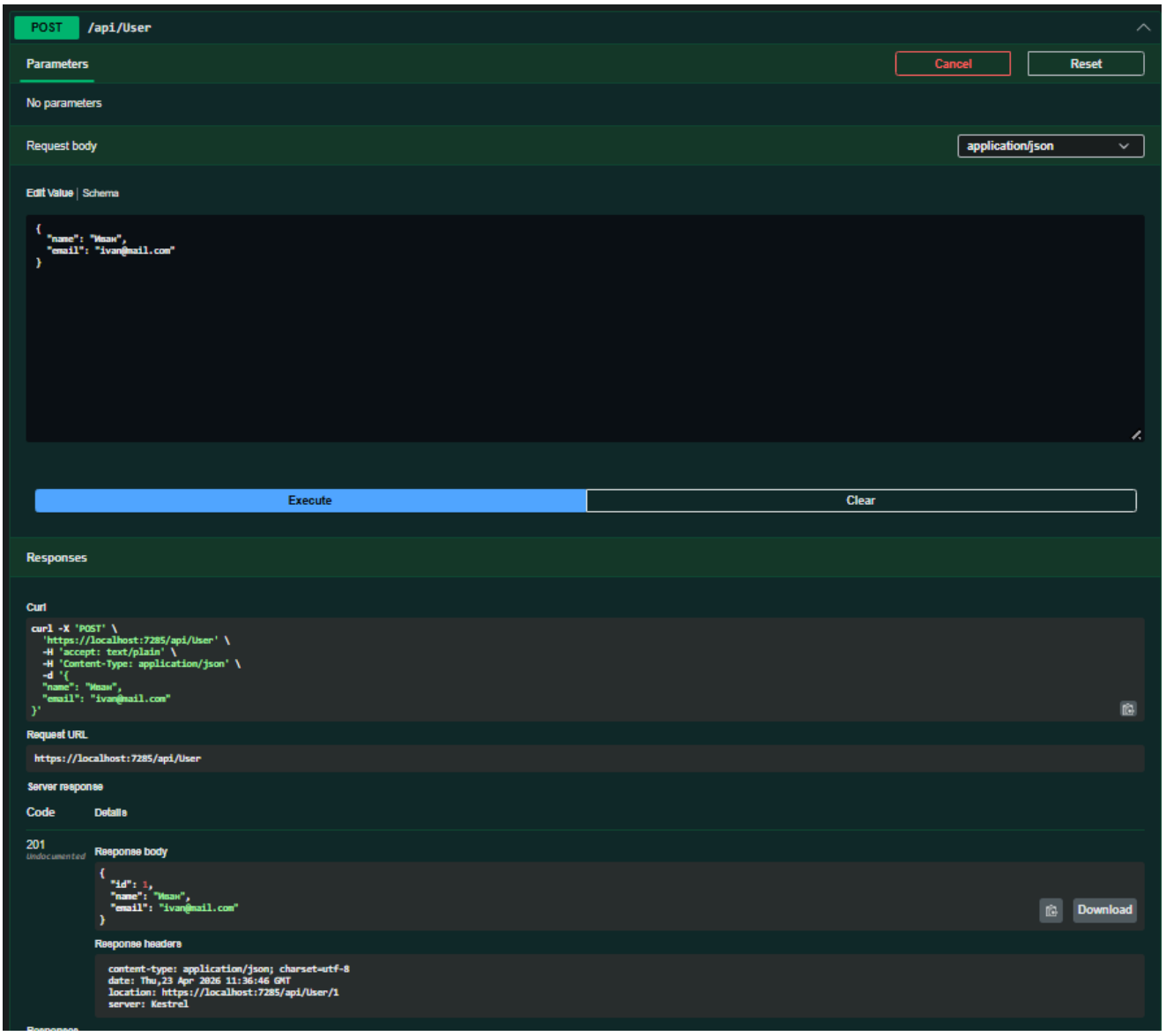


Swagger UI покажет сформированный запрос, реальный ответ сервера с кодом состояния и телом, а также заголовки ответа.



Отправка тела запроса

Для POST и PUT эндпоинтов Swagger UI генерирует форму с примером тела запроса на основе структуры модели. Отредактируйте JSON и нажмите **Execute**.



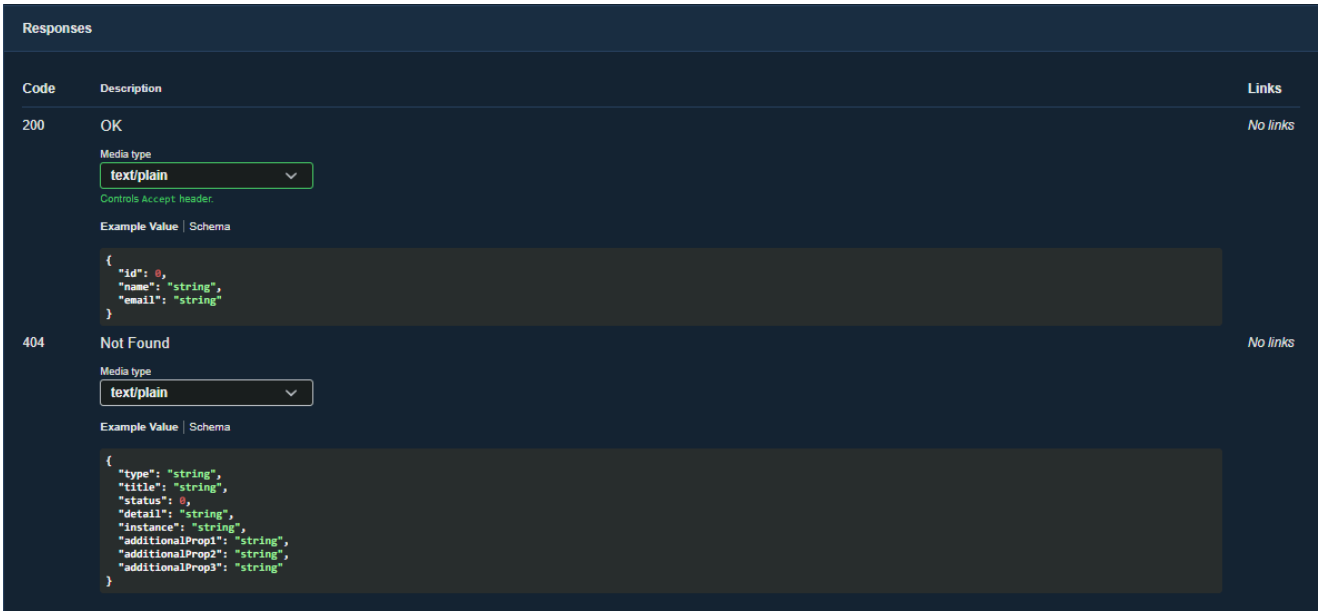
Документирование возможных ответов

По умолчанию Swagger показывает только ответ `200`. Если метод может вернуть `404` или `400` — добавьте атрибут `[ProducesResponseType]`:

```
[HttpGet("{id}")]
[ProducesResponseType(typeof(User), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public ActionResult<User> GetById([FromRoute] int id)
{
    var user = _users.FirstOrDefault(u => u.Id == id);
    return user is null ? NotFound() : Ok(user);
}

[HttpPost]
[ProducesResponseType(typeof(User), StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
public ActionResult<User> Create([FromBody] UserDto dto)
{
    if (string.IsNullOrWhiteSpace(dto.Name))
```

```
        return BadRequest("Имя пользователя обязательно.");

    var user = new User(_nextId++, dto.Name.Trim(), dto.Email);
    _users.Add(user);

    return CreatedAtAction(nameof(GetById), new { id = user.Id }, user);
}
```

Теперь каждый эндпоинт в Swagger UI показывает все возможные ответы со схемами данных.



Описания через XML-комментарии

Swagger умеет читать XML-комментарии к методам и моделям и отображать их в документации. Это наиболее быстрый способ сделать API понятным без написания отдельной документации.

Включается в два шага.

Шаг 1. Включите генерацию XML-документации в файле проекта `.csproj`:

```
<PropertyGroup>
    <GenerateDocumentationFile>true</GenerateDocumentationFile>
    <NoWarn>$(NoWarn);1591</NoWarn>
</PropertyGroup>
```

Строка `<NoWarn>1591</NoWarn>` подавляет предупреждения компилятора о публичных членах без XML-комментариев — без неё проект будет выдавать сотни предупреждений для всех незадокументированных методов.

Шаг 2. Подключите XML-файл в конфигурации Swagger в `Program.cs`:

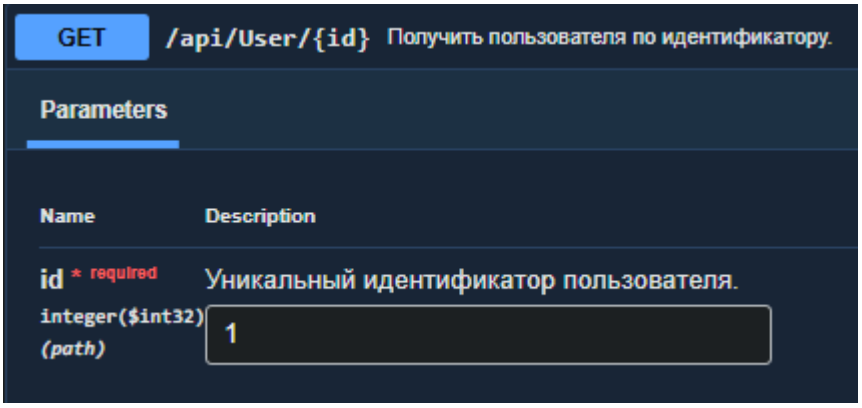
```
builder.Services.AddSwaggerGen(options =>
{
    var xmlFile = $"{Assembly.GetExecutingAssembly().GetName().Name}.xml";
    var xmlPath = Path.Combine(AppContext.BaseDirectory, xmlFile);
    options.IncludeXmlComments(xmlPath);
});
```

Не забудьте добавить `using System.Reflection;` в начало файла.

Теперь XML-комментарии над методами и моделями отображаются в документации:

```
/// <summary>
/// Получить пользователя по идентификатору.
/// </summary>
/// <param name="id">Уникальный идентификатор пользователя.</param>
[HttpGet("{id}")]
[ProducesResponseType(typeof(User), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public ActionResult<User> GetById([FromRoute] int id)
{
    var user = _users.FirstOrDefault(u => u.Id == id);
    return user is null ? NotFound() : Ok(user);
}
```

```
/// <summary>Данные для создания или обновления пользователя.</summary>
public record UserDto(
    /// <summary>Имя пользователя. Не может быть пустым.</summary>
    string Name,
    /// <summary>Адрес электронной почты.</summary>
    string Email);
```



Итоговая конфигурация

Соберём все настройки вместе — финальный вид `Program.cs` для учебного проекта:

```
using System.Reflection;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    var xmlFile = $"{Assembly.GetExecutingAssembly().GetName().Name}.xml";
    var xmlPath = Path.Combine(AppContext.BaseDirectory, xmlFile);
    options.IncludeXmlComments(xmlPath);
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```